

Cours 03 — Les dépôts distants et GitHub

Module 3/8 — Formation Git & GitHub

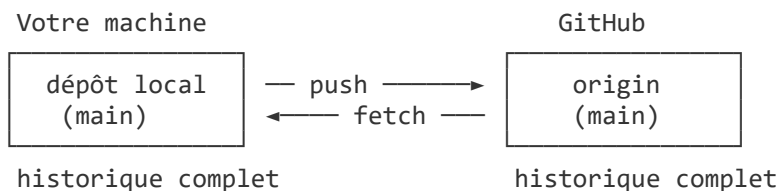
Objectifs

À la fin de ce module, vous saurez : - expliquer la relation entre votre dépôt **local** et un dépôt **distant** (*remote*), et ce que sont les **branches de suivi** (*origin/main*) ; - cloner un dépôt, envoyer vos commits (*push*) et récupérer ceux des autres (*fetch*, *pull*) ; - distinguer *fetch* (sans risque) de *pull* (qui modifie votre branche) ; - configurer l'authentification **SSH** ou **HTTPS + jeton** avec GitHub, et réagir à un *push* rejeté.

Théorie

Local et distant : deux dépôts complets, pas un serveur

Git est **distribué** (cours 01) : le dépôt sur GitHub n'est pas « le vrai » dépôt dont le vôtre serait une vue partielle. C'est un dépôt **complet**, exactement comme le vôtre, qui sert de **point de rendez-vous** entre plusieurs copies. Par convention, ce dépôt distant s'appelle *origin*.



Les branches de suivi : *origin/main*

Votre dépôt local garde une **photographie** de l'état du distant, sous forme de branches spéciales appelées **branches de suivi** : *origin/main*, *origin/develop*... Point crucial : ces photographies **ne se mettent à jour que lorsque vous communiquez avec le distant** (*fetch* ou *pull*). Entre deux *fetch*, *origin/main* peut être périmé sans que vous le sachiez.

```
local      : C1 — C2 — C3          ← main (votre travail)
origin/    : C1 — C2                ← origin/main (dernière photo connue du
distant)
```

Ici votre *main* a un commit d'avance sur la dernière photo du distant : un *push* l'enverra. Mais quelqu'un a peut-être poussé un C-autre entretemps — vous ne le verrez qu'après un *fetch*.

Les trois verbes du distant

Commande	Effet	Risque
<i>fetch</i>	Télécharge les nouveautés et met à jour <i>origin/*</i> — sans toucher à vos branches	Aucun
<i>pull</i>	<i>fetch</i> puis merge (ou rebase) dans votre branche courante	Peut créer un conflit
<i>push</i>	Envoie vos commits locaux vers le distant	Peut être rejeté (voir plus bas)

La bonne habitude : *fetch* d'abord pour **regarder** ce qui est arrivé, puis décider. *pull* est la version pressée (*fetch* + merge d'un coup), pratique mais qui agit sans que vous ayez inspecté.

SSH ou HTTPS ? Choisir une fois pour toutes

GitHub n'accepte **plus** votre mot de passe de compte pour `git push`. Il faut donc l'un des deux :

- **HTTPS + jeton (PAT)**. L'URL commence par `https://`. Au premier push, on fournit un *Personal Access Token* (jeton personnel) à la place du mot de passe. Simple, marche partout, y compris derrière un pare-feu d'entreprise.
- **SSH**. L'URL commence par `git@github.com:`. On génère **une paire de clés** (une privée qui reste sur la machine, une publique qu'on colle sur GitHub). Plus rien à ressaisir ensuite — le plus confortable au quotidien.

Il n'y a pas de mauvais choix. SSH pour votre machine perso, HTTPS + jeton si SSH est bloqué : les deux mènent au même GitHub.

Commandes commentées

Cloner — `git clone`

```
git clone git@github.com:utilisateur/gestionnaire-ordres.git
```

Récupère un dépôt existant : crée le dossier, télécharge tout l'historique, configure `origin` automatiquement et bascule sur `main`. C'est la commande de départ quand le projet existe déjà sur GitHub (le vôtre ou celui d'un autre).

Relier un dépôt local à GitHub — `git remote`

```
git remote add origin git@github.com:utilisateur/gestionnaire-ordres.git
git remote -v                                # vérifier les distants configurés (fetch + push)
```

À utiliser dans l'autre sens : vous avez déjà un dépôt local (créé au cours 01 avec `git init`) et vous venez de créer un dépôt **vide** sur GitHub. `remote add origin` colle l'adresse du distant sur le nom `origin`. `git remote -v` liste ce qui est configuré — toujours utile pour vérifier qu'on pousse au bon endroit.

Premier envoi — `git push -u`

```
git push -u origin main                # premier push : -u LIE main à origin/main
git push                               # ensuite, « git push » tout court suffit
```

Le `-u` (`--set-upstream`) établit le lien entre votre branche `main` et `origin/main`. Une fois ce lien posé, Git sait où pousser et où tirer : `git push` et `git pull` sans argument suffisent. On ne met `-u` qu'**une seule fois par branche**, au tout premier push.

Récupérer sans risque — `git fetch`

```
git fetch                                # met à jour origin/* ; ne touche PAS à vos branches
git log main..origin/main --oneline     # « qu'est-ce qui est arrivé que je n'ai pas ? »
```

`fetch` est l'opération **toujours sans danger** : elle rafraîchit la photographie `origin/*` mais laisse votre travail intact. La deuxième ligne montre les commits présents sur `origin/main` mais absents de votre `main` : c'est ce que vous récupérerez en fusionnant.

Récupérer et intégrer — `git pull`

```
git pull                                # = fetch + merge dans la branche courante
git pull --rebase                        # = fetch + rebase : historique linéaire, sans commit de fusion
```

`pull` enchaîne `fetch` puis intègre. Par défaut il **fusionne** (merge), ce qui peut créer un commit de fusion. `--rebase` rejoue plutôt vos commits locaux **par-dessus** ce qui est arrivé : l'historique reste une ligne droite (voir cours 05). Beaucoup d'équipes préfèrent `--rebase` pour un historique plus lisible.

Configurer SSH — `ssh-keygen`

```
ssh-keygen -t ed25519 -C "votre.email@exemple.com" # générer la paire de clés
cat ~/.ssh/id_ed25519.pub # afficher la clé PUBLIQUE (à coller sur GitHub)
ssh -T git@github.com # tester : « Hi <vous>! You've successfully
authenticated »
```

ssh-keygen crée deux fichiers dans ~/.ssh/ : id_ed25519 (**privée** — ne la partagez jamais, ne la committez jamais) et id_ed25519.pub (**publique** — c'est celle-ci qu'on colle dans *GitHub* → *Settings* → *SSH and GPG keys*). Le -C n'est qu'un commentaire pour reconnaître la clé. ssh -T valide que tout est branché avant le premier push.

Quand push est rejeté

```
git push
# ! [rejected] main -> main (non-fast-forward)
```

```
git pull --rebase # récupérer et rejouer son travail par-dessus
# ... résoudre d'éventuels conflits ...
git push # repousser : cette fois ça passe
```

Un push rejeté avec non-fast-forward veut dire une seule chose : **le distant a avancé depuis votre dernière synchro** (quelqu'un a poussé, ou vous depuis une autre machine). Git refuse d'écraser ce travail. La solution est **toujours** la même : récupérez d'abord (pull --rebase), réglez les éventuels conflits, puis repoussez. **Ne forcez jamais** (git push --force) sur une branche partagée : vous détruiriez les commits des autres.

Exercices pratiques

1. **Création et premier push.** Créez un dépôt **vide** sur GitHub, reliez-y votre dépôt local du cours 01 (git remote add origin ...) et poussez-le avec git push -u origin main.
2. **Aller-retour.** Modifiez un fichier directement dans l'interface web de GitHub (bouton crayon ) , puis récupérez le changement en local : d'abord git fetch (observez que vos fichiers ne bougent **pas**), puis git pull (là ils changent). Mesurez la différence.
3. **Divergence.** Committez en local **et** via l'interface web sur la même branche, puis tentez un git push. Constatez le rejet, puis résolvez avec git pull --rebase et repoussez.
4. **SSH.** Générez une clé SSH, ajoutez la clé **publique** sur GitHub, et validez la connexion avec ssh -T git@github.com.

Solutions des exercices

Solution 1 — Création et premier push

Sur GitHub : bouton **New repository**, donnez-lui un nom, **ne cochez rien** (pas de README, pas de .gitignore — le dépôt doit être vide pour accueillir le vôtre). Puis en local :

```
cd demo-git # votre dépôt du cours 01
git remote add origin git@github.com:VOTRE-PSEUDO/demo-git.git
git remote -v # vérifier l'URL

git push -u origin main # envoie l'historique ; -u lie main à
origin/main
```

Rechargez la page GitHub : vos fichiers et vos commits y sont. Si Git réclame master au lieu de main, c'est que votre branche s'appelle master — renommez-la avec git branch -M main avant de pousser.

Solution 2 — Aller-retour (fetch vs pull)

Sur GitHub, ouvrez un fichier, cliquez le crayon  , changez une ligne, **Commit changes**. Puis en local :

```
git fetch
git status
# → « Your branch is behind 'origin/main' by 1 commit »
#   MAIS vos fichiers sur le disque n'ont PAS changé : fetch ne fait que la photo.
```

```
git log main..origin/main --oneline    # voir le commit web qu'on n'a pas encore
```

```
git pull                                # MAINTENANT le fichier local reçoit la
modification
```

La leçon : fetch vous **informe** (la photo origin/main avance), pull **agit** (votre branche et vos fichiers changent).

Solution 3 — Divergence et push rejeté

```
# 1) Un commit LOCAL
echo "modif locale" >> notes.md && git add . && git commit -m "feat: modif locale"
```

```
# 2) Un commit WEB sur la MÊME branche (crayon ✎ sur GitHub, sur un autre fichier
ou la même ligne)
```

```
# 3) Tenter de pousser
```

```
git push
# ! [rejected] ... (non-fast-forward) ← le distant a avancé de son côté
```

```
git pull --rebase                        # récupère le commit web, rejoue le vôtre par-
dessus
```

```
# (si conflit : éditer, git add, puis git rebase --continue – voir cours 05)
```

```
git push                                # cette fois, accepté
git log --oneline --graph                # historique propre, linéaire grâce au --rebase
```

Solution 4 — SSH

```
ssh-keygen -t ed25519 -C "votre.email@exemple.com"
# Entrée pour accepter le chemin par défaut ; passphrase facultative (Entrée pour
aucune)
```

```
cat ~/.ssh/id_ed25519.pub                # copier TOUTE la ligne affichée (commence par
ssh-ed25519)
```

Sur GitHub : *Settings* → *SSH and GPG keys* → *New SSH key*, collez la clé **publique**, enregistrez. Puis testez :

```
ssh -T git@github.com
# → Hi VOTRE-PSEUDO! You've successfully authenticated...
```

Si vos dépôts étaient clonés en HTTPS, basculez-les en SSH avec `git remote set-url origin git@github.com:VOTRE-PSEUDO/depot.git`.

Points clés à retenir

- Le distant (origin) est un dépôt **complet**, un point de rendez-vous — pas un serveur qui détiendrait « la » vérité.
- origin/main est une **photographie locale** du distant : elle n'est à jour qu'après un fetch.
- `git fetch` est **toujours** sans risque ; `git pull` **modifie** votre branche courante.
- `git push -u` la première fois pour lier la branche, puis `git push` tout court.

- Un push rejeté = le distant a avancé : `git pull --rebase`, puis repoussez. **Jamais** `--force` sur une branche partagée.